

# Design, Integration and PLC Coordination of a Multi-Robot Semiconductor Wafer Handling Cell

Prajjwal Dutta

*Ira A. Fulton Schools of Engineering*

*Arizona State University*

Tempe, AZ, USA

pdutta9@asu.edu

**Abstract:** This paper reports the design, re-implementation and integration of a laboratory-scale semiconductor wafer handling cell in which a four-axis SCARA robot, a six-degree-of-freedom collaborative robot, a stepper-driven conveyor with camera inspection, and an Allen-Bradley Micro820 programmable logic controller cooperate to move 100 mm silicon wafers from a stacked cassette, through an inspection station, onto a deposit platform. The cell was originally commissioned on a Dobot M1 running the `dType` Python API; the work described here re-hosts the wafer extraction stage on a Dobot M1 Pro under DobotStudio Pro 2.8.3, where the program language is Lua and the vendor API exposes no explicit arm-configuration selector. The central technical result is that the deposit fixture lies at 393.8 mm of the arm's 400 mm reach, where only one inverse-kinematic elbow branch admits a solution, so purely Cartesian commanding is rejected by the controller; the adopted remedy commands the rack side in Cartesian coordinates, where a new shelf height must be computed each cycle, and the fixture side in joint coordinates, where the branch is carried implicitly by the taught joint tuple. A second result concerns the coordination layer: the as-built seven-rung Micro820 ladder is shown to differ materially from the program published in the course manual: it gates every rung on a latched `Start`, adds a stop input, uses IEC `TP` pulse timers rather than retentive on-delay timers, and drives the two conveyor outputs through negated coils. A dedicated handshake rung set and a structured-text sequencer with belt watchdogs were developed for the new SCARA. Polarity of the panel stop button was measured normally-closed, which inverts its contact form on every rung and had silently prevented the start rung from conducting. Fifteen documented faults, their root causes and their resolutions are tabulated.

**Index Terms:** industrial automation, SCARA kinematics, collaborative robots, programmable logic controllers, ladder logic, machine vision, wafer handling, system integration

## I. INTRODUCTION

SEMICONDUCTOR front-end handling is an unusually demanding automation problem for its physical scale. A 100 mm wafer weighs only a few tens of grams, but it is thin, brittle, and must not be contacted on its polished face or clamped hard on its edge. The interesting engineering is therefore not payload or speed but geometry, sequencing and the discipline with which independent machines are made to agree about whose turn it is to occupy a shared workspace.

The cell described here was built at Arizona State University as the hardware platform for a five-laboratory short course in robot integration [1]. Its material flow is simple to state: a SCARA robot draws a wafer from a vertical cassette and lays

it in a cradle on a conveyor; the conveyor carries the cradle past a camera that photographs the wafer and confirms its presence; a collaborative robot lifts the wafer from the cradle and places it on a deposit platform; the conveyor returns and the cycle repeats one shelf lower. Every stage is triggered by, and reports to, a Micro820 PLC acting as the cell-level coordination layer.

Reproducing that flow on the hardware actually available proved to be the substance of the work. Three of the five subsystems differ from the ones the original commissioning targeted:

- the SCARA is a Dobot **M1 Pro** programmed in **Lua** under DobotStudio Pro 2.8.3, not a Dobot M1 driven from Python through `DobotDllType`;
- the collaborative-robot end effector is a **vacuum** tool on a 24 V solenoid manifold rather than the DH PGE 8 electric parallel gripper with printed finger extensions;
- the fixtures were re-taught after the rack was physically moved to keep the shoulder joint inside its limit, so every absolute coordinate in the original program is invalid on this cell.

None of these is a like-for-like substitution. The API change removes the explicit arm-configuration call that the original program relied on; the fixture move changes the insertion axis from *Y* to *X*; the tool change converts a two-state gripper into a pneumatic circuit with its own vacuum switch. The contributions reported in this paper are:

- 1) A hybrid Cartesian/joint-space motion formulation that makes the SCARA's elbow configuration explicit where the workspace is ill-conditioned and parametric where a new height must be computed each cycle (Section III).
- 2) A closed reading of the as-built Micro820 ladder from the project source, an accounting of the ways it departs from the published program, and the electrical measurement that explains a start rung which refused to conduct (Section VII).
- 3) A cell-integration surface for the SCARA (start gate, cycle-done pulse, busy and run-done outputs), designed so that a broken wire fails towards *no request* rather than towards continuous operation, together with the matching PLC rung set and a structured-text sequencer with belt watchdogs (Sections VII-F and VII-H).
- 4) A defensive program structure in which every com-

TABLE I  
CELL HARDWARE AND ACCESS PATHS

| Element      | Controller                                                   | Access                                                                        |
|--------------|--------------------------------------------------------------|-------------------------------------------------------------------------------|
| SCARA        | Dobot M1 Pro, 4 axes,<br>400 mm reach                        | DobotStudio Pro 2.8.3,<br>Lua 5.3.5, Ethernet<br>192.168.1.6                  |
| Cobot        | Elephant Robotics<br>Pro600, 6 DoF,<br>600 mm, 3 kg          | RoboFlow on onboard<br>Linux; RealVNC /<br>pymycobot at<br>192.168.1.159:5001 |
| Conveyor     | Dobot Magic Box<br>extended I/O, stepper<br>on EMotor port 0 | Python<br>DobotDllType over<br>COM3 at<br>115 200 baud                        |
| Inspection   | USB camera, index 0                                          | OpenCV<br>HoughCircles                                                        |
| Coordination | Allen-Bradley<br>Micro820<br>2080-LC20-20QBB                 | Connected<br>Components<br>Workbench,<br>AB_ETHIP-1<br>\169.254.19.93         |

manded pose is range-checked before it is issued, untaught placeholder poses abort rather than move, and a fault leaves the arm where it stands (Section III-F).

## II. CELL ARCHITECTURE

### A. Material Flow and Layout

Figure 2 shows the bench as built. The SCARA stands at one end with the wafer cassette immediately in front of it; the conveyor runs down the length of the bench with the wafer cradle taped to the belt; the collaborative robot stands at the far end with the deposit platform inside its envelope. A camera on a flexible arm views the belt between the two robots, and an infrared photoelectric sensor sits at each end of the belt.

### B. Devices, Networks and Power

Table I lists the controllers and their access paths. Four independent programming environments are involved, and no two of them share a language. This is characteristic of small-cell integration and is the reason the PLC's role is deliberately narrow: it is the only element that sees the whole cell, so it is kept to signalling rather than motion.

The Micro820 address is link-local, so the programming PC's network adapter must be configured on 169.254.x.x/255.255.0.0 before the controller will appear in the browse tree. This is a recurring practical obstacle and is noted here because it presents as a dead controller rather than as a network misconfiguration.

Power is split. The M1 Pro runs from a 48 V supply, the PLC and the sensors from 24 V. Every signal crossing between them passes through an eight-channel opto-isolated interface board. Isolation is not a refinement here: it removes the need for the two grounds to be bonded, and it makes the sinking or sourcing convention of each device irrelevant at the boundary.

TABLE II  
M1 PRO LIMITS USED FOR PRE-FLIGHT CHECKING

| Axis / quantity | Range           | Note                                |
|-----------------|-----------------|-------------------------------------|
| $J_1$ shoulder  | $\pm 85^\circ$  | the binding limit in practice       |
| $J_2$ elbow     | $\pm 130^\circ$ | sign selects the branch             |
| $J_3$ screw     | 5 to 245 mm     | software limit; $J_3 \equiv Z$      |
| $J_4$ wrist     | $\pm 360^\circ$ | $R$ defined only modulo $360^\circ$ |
| Outer reach     | 400 mm          | $\sqrt{X^2 + Y^2}$                  |
| Inner dead zone | 153 mm          | arm cannot fold inside              |

## III. SCARA WAFER EXTRACTION

### A. Kinematics of the M1 Pro

The M1 Pro is a SCARA with two revolute links in the horizontal plane, a vertical lead screw and a wrist. Its third axis is linear, so  $J_3$  is  $Z$  in millimetres, a convenience with real consequences, since a change in  $J_3$  alone is a pure vertical translation that cannot sweep the tool into anything. With  $L_1 = L_2 = 200$  mm from the dimension drawing [2], forward kinematics are

$$X = L_1 \cos J_1 + L_2 \cos(J_1 + J_2) \quad (1)$$

$$Y = L_1 \sin J_1 + L_2 \sin(J_1 + J_2) \quad (2)$$

$$Z = J_3, \quad R = J_1 + J_2 + J_4 \pmod{360^\circ}. \quad (3)$$

The model was validated against live panel data before it was used for anything: joint readings  $J_1 = 19.02^\circ$ ,  $J_2 = 29.37^\circ$ ,  $J_4 = -159.79^\circ$  substituted into (1) to (3) return (321.9, 214.7) with  $R = -111.40^\circ$ , reproducing the taught rack approach point to the decimal. Table II records the joint and envelope limits used throughout.

### B. Fork End Effector and Cassette Geometry

The M1 Pro carries a bare tool flange. A 3D-printed fork with two thin prongs bolts directly to it (Fig. 3). The fork slides into a cassette slot, passes under the wafer, and the  $Z$  screw lifts the wafer off its shelf. There is no clamping and no vacuum on this side of the cell: the wafer simply rests on the prongs during transfer. This is the gentlest of the available strategies and it is why the SCARA half of the cycle tolerates comparatively coarse positioning. It also means the deposit is not complete until the prongs have been withdrawn along their own axis, a point returned to in Section III-E.

The cassette is the manual's rack, so its internal geometry is a specification rather than a measurement: 20 mm shelf pitch with the floor at  $Z = 30$  mm. That settles the shelf-stepping constant, and the program refuses to run more than one cycle until the value has been confirmed physically. The measured lift of 13.0 mm is consistent with a 20 mm pitch: it clears the wafer's own shelf lip while staying comfortably below the shelf above.

### C. The Elbow-Branch Problem

A Cartesian pose  $(X, Y, Z, R)$  does not determine a SCARA configuration. For a two-link planar arm, a reachable  $(X, Y)$  at radius  $\rho = \sqrt{X^2 + Y^2}$  admits two solutions,

# PLC-Controlled Wafer Cell: Wiring Overview

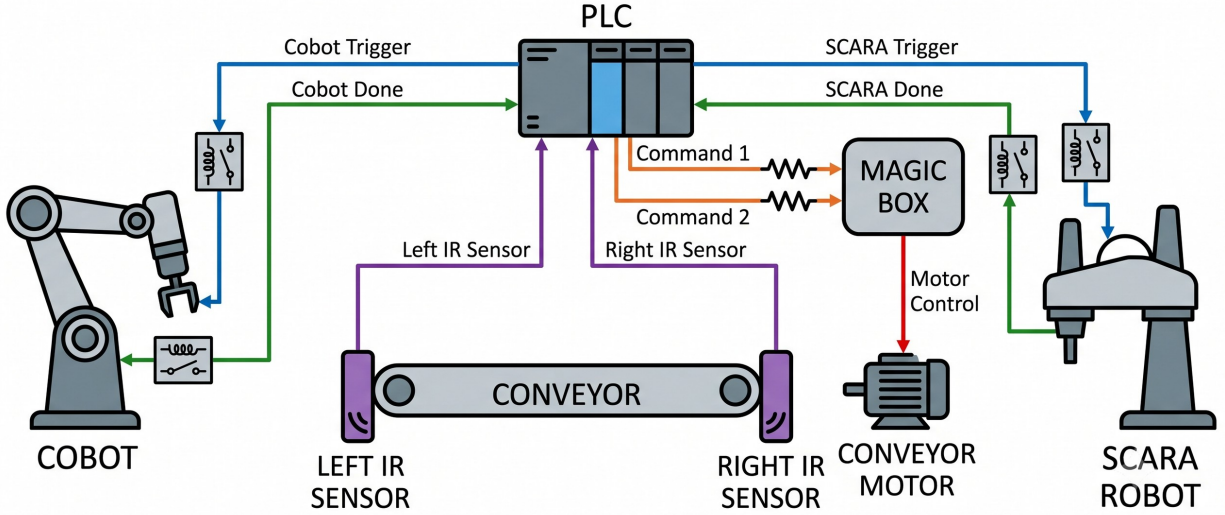


Fig. 1. Signal architecture of the cell. The Micro820 is a coordination layer only: it issues trigger pulses and receives done pulses, but it does not drive the conveyor motor or either arm directly. Every crossing between the PLC and a robot passes through an opto-isolated interface board, so the 24 V control ground and the 48 V arm ground never meet.

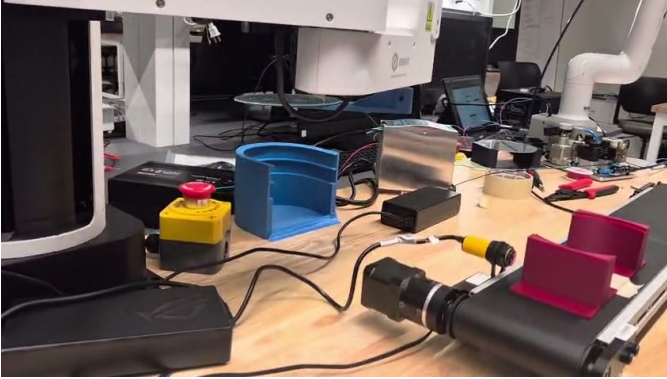


Fig. 2. The assembled cell during a run. Left: the M1 Pro and the blue wafer cassette. Centre: the conveyor, its 3D-printed wafer cradle and the end-of-travel infrared sensor. Right: the Pro600 and the deposit platform.



Fig. 3. The Dobot M1 Pro with the 3D-printed two-prong fork fitted to the  $J_4$  flange. The deposit platform and a spare cradle are on the bench in the foreground.

$$\cos J_2 = \frac{\rho^2 - L_1^2 - L_2^2}{2L_1L_2}, \quad J_2 = \pm \arccos(\cdot), \quad (4)$$

distinguished by the sign of  $J_2$ , the “left” and “right” elbow. Both are legitimate; the controller’s solver picks one.

The deposit fixture sits at  $\rho = 393.8$  mm of a 400 mm reach, i.e. within 1.5% of full extension. Two things happen there. First, the Jacobian becomes ill-conditioned: small Cartesian increments demand large joint excursions, so positioning accuracy degrades and the arm moves in ways that look disproportionate to the command. Second, and decisively, only

one branch remains inside the joint limits. Table III works the point that was originally rejected on hardware.

An earlier analysis had rejected *both* branches by treating  $J_4 = -226.3^\circ$  as beyond the wrist limit. That was wrong:  $J_4$  on the M1 Pro spans  $\pm 360^\circ$ , and  $-226.3^\circ$  is the same wrist orientation as  $+133.7^\circ$  because  $R$  in (3) is defined only modulo  $360^\circ$ . Correcting the limit changes the diagnosis in a way that matters: the point is not unreachable, it is reachable

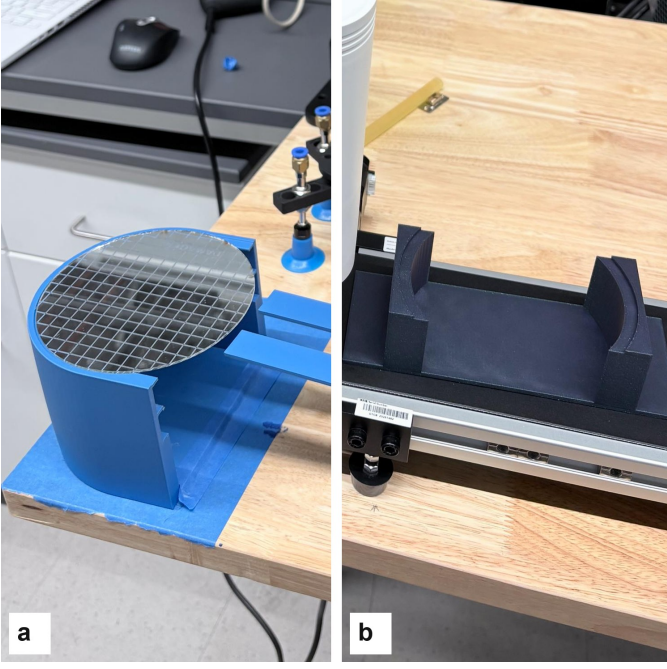


Fig. 4. Printed fixtures as built. Both were redesigned after the model of Fig. 10 was prepared, so these photographs, not that model, are the current geometry. (a) The wafer cassette with a 100 mm silicon wafer on the top shelf; the fork prongs enter from the front, and the twin vacuum cups of the cobot tool are visible at the right edge. (b) The wafer cradle mounted on the conveyor belt: two curved arms that locate the wafer edge without contacting its face.

TABLE III  
BOTH IK BRANCHES FOR THE TRAVERSE POINT (113.3, 214.7),  
 $R = -111.4^\circ$

| Branch      | $J_1$   | $J_2$   | $J_4$   | Verdict                      |
|-------------|---------|---------|---------|------------------------------|
| A (elbow +) | +9.54   | +105.27 | +133.79 | within all limits            |
| B (elbow -) | +114.81 | -105.27 | -120.94 | $J_1$ exceeds $\pm 85^\circ$ |

on exactly one branch, and the controller was solving for the other one. That explanation also accounts for an observation which the “unreachable” theory did not: the operator can reach the fixture by hand-guiding but not by Cartesian jogging, because hand-guiding moves joints directly and never asks the solver to choose.

Three remedies were considered. *Substituting linear interpolation* was rejected: a straight-line move requires every intermediate point to be solvable, which is strictly more constrained than a joint-interpolated move, so it fails harder rather than more gracefully. *Selecting the configuration explicitly*, as the original M1 program did with `SetArmOrientation`, was unavailable, because the Lua API exposes equivalent. *Commanding the fixture side in joint coordinates* was adopted. A taught joint tuple *is* a configuration; there is no solve and therefore no ambiguity. The rack side stays Cartesian because a new  $Z$  must be computed for every shelf, and parametric Cartesian is the natural expression of that.

#### D. Taught Poses

Five poses were taught by hand-guiding and read back from the controller with a probe script rather than transcribed from the pendant, which removes an entire class of digit errors. Table IV lists them with the Cartesian equivalents computed from (1) to (3).

Three properties of Table IV are worth stating because they are checks on the teaching rather than incidental.

*Every pose has  $J_2 > 0$ .* All five are on branch A, the same branch as the rack side, so the arm never reconfigures anywhere in the cycle. A pose returning branch B would require a mid-path flip and was grounds for re-teaching rather than running.

*Four poses share  $Y = 363.4$  mm.* The traverse, approach, deposit and withdrawal poses lie on one line, which is what a straight fork withdrawal along  $X$  should produce.  $J\_ABOVE$  and  $J\_DEPOSIT$  differ only in  $J_3$ : a pure vertical descent onto the fixture.  $J\_WITHDRAWN$  retracts from  $X = 134.4$  to  $X = 4.2$  mm at unchanged  $Y$ , a 130.2 mm translation along the prong axis.

*The home pose sits directly above the rack approach point.* Taught home is (109.6, 352.7) at  $Z = 187.8$ ; the rack approach is (109.6, 352.7) at the shelf height. Home is therefore the approach point lifted to clear height, which is why the descent into the cassette is a pure  $Z$  move.

The factory home was explicitly *not* used. All-joints-zero puts the arm straight out at  $X = 400$ ,  $Y = 0$  with  $J_3$  at the bottom of the screw, below the 5 mm software limit and with the fork fully extended across the bench at minimum height.

#### E. The Cycle

Table V gives the seven steps of one wafer and the coordinate system each is commanded in.

Insertion is divided into thirds rather than issued as a single move. The intermediate points cost nothing in cycle time worth counting and they give the operator three chances to see the prongs entering off-centre before the fork is deep in a slot.

Step 6 requires two moves, not one, and this is physical rather than stylistic. Lowering the wafer onto the cradle leaves the fork still underneath it. The prongs must slide out along their own axis before the arm goes anywhere else; any other motion from the deposit pose drags the wafer off the cradle with the tool. Depositing is not finished until the fork is clear, which is why the wafer counter increments after  $J\_WITHDRAWN$  and not after  $J\_DEPOSIT$ . An aborted cycle can therefore never inflate the count.

Shelf heights are derived from the wafer count rather than the reverse:

$$z_n = z_0 - (f + n - 2) \Delta, \quad n = 1 \dots N, \quad (5)$$

with  $z_0 = 85.4$  mm the top shelf,  $f$  the first loaded shelf index,  $\Delta = 20$  mm the pitch and  $N$  the number of wafers loaded. The earlier formulation stepped  $Z$  downwards until it ran out, which on this cassette produced *four* cycles for three wafers, the fourth driving the fork at an empty slot. Geometry

TABLE IV  
TAUGHT JOINT POSES AND THEIR FORWARD-KINEMATIC EQUIVALENTS. ALL FIVE LIE ON ELBOW BRANCH A ( $J_2 > 0$ ), SO THE ARM NEVER RECONFIGURES DURING A CYCLE.

| Pose        | $J_1$ | $J_2$ | $J_3$  | $J_4$  | $X$   | $Y$   | $Z$   | $R$    |
|-------------|-------|-------|--------|--------|-------|-------|-------|--------|
| J_HOME      | 50.16 | 45.17 | 187.82 | 103.37 | 109.6 | 352.7 | 187.8 | -161.3 |
| J_TRAVERSE  | 63.65 | 49.28 | 166.69 | 267.51 | 10.8  | 363.4 | 166.7 | 20.4   |
| J_ABOVE     | 55.33 | 28.74 | 166.69 | 296.37 | 134.4 | 363.4 | 166.7 | 20.4   |
| J_DEPOSIT   | 55.33 | 28.74 | 89.19  | 296.37 | 134.4 | 363.4 | 89.2  | 20.4   |
| J_WITHDRAWN | 64.64 | 49.38 | 76.53  | 266.42 | 4.2   | 363.4 | 76.5  | 20.4   |

TABLE V  
ONE WAFER CYCLE

| # | Step                    | Frame     | Pose(s)                |
|---|-------------------------|-----------|------------------------|
| 1 | Home                    | joints    | J_HOME                 |
| 2 | Cassette approach       | Cartesian | $x_0, y_0, z_n, r_0$   |
| 3 | Fork inserted           | Cartesian | $-X$ in three steps    |
| 4 | Wafer lifted, withdrawn | Cartesian | $+Z$ , then $+X$ out   |
| 5 | Traverse to fixture     | joints    | J_TRAVERSE, J_ABOVE    |
| 6 | Deposit, prongs clear   | joints    | J_DEPOSIT, J_WITHDRAWN |
| 7 | Return home             | joints    | J_TRAVERSE, J_HOME     |

alone cannot catch that error, because a four-wafer plan is perfectly legal on this cassette; only the loaded count knows how many wafers there are. Making the count authoritative and  $Z$  derived removes the failure mode by construction. The floor guard survives, but now for the opposite job: it aborts if a count walks past the bottom of the cassette.

#### F. Guards and Staged Homing

Every commanded motion passes through one of two wrappers. The Cartesian wrapper rejects a target whose radius exceeds the reach or falls inside the dead zone, or whose  $Z$  leaves the screw range. The joint wrapper rejects a tuple outside any joint limit and, separately, rejects a tuple still at its  $\{0, 0, 0, 0\}$  placeholder. That second check matters more than it looks: all-zeros is not an inert sentinel but a real pose, arm straight out at the bottom of the screw, and commanding it would drive the fork through whatever stands in front of the robot. Once any check fails, an abort flag suppresses every subsequent move, so a failure cannot leave the arm half-inserted and then continue.

The first move of a cycle is a joint move to home from wherever the arm happens to be standing, and joint interpolation moves all four axes together along a path the endpoints do not describe. Starting that from inside a cassette slot drags the prongs sideways through a shelf. The program therefore reads the live joint angles before commanding anything and, if the arm is not already near home, approaches home in three stages:

- 1) if forward kinematics place the tool inside the cassette corridor, back straight out along the insertion axis at the current height, which is the only safe exit from a slot, since rising would strike the shelf above;
- 2) rise to clear height with  $X$ ,  $Y$  and  $R$  held, which is a pure screw move and cannot sweep into anything;

TABLE VI  
API MIGRATION, DTYPE (PYTHON) TO DOBOTSTUDIO PRO (LUA)

| MIStudio / Python                                                                                                                         | DobotStudio Pro / Lua                                                                                                  |
|-------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| SetPTPCmdSync(<br>api, l, x, y, z, r)<br>(none)                                                                                           | MovJ(P, {... SYNC=1}), P<br>Cartesian<br>JointMovJ(P, {...}),<br>P={joint={...}}<br>SpeedFactor() plus SpeedJ/<br>AccJ |
| SetPTPCCommonParams(<br>api, l0, l1, l)<br>GetIODI(api, l8) [0]<br>SetIODOSync(api, l8, l)<br>dSleep(500)<br>SetArmOrientation(api, 0, l) | DI(l8) returns ON/OFF<br>DO(l8, ON)<br>Sleep(500)<br><b>no equivalent</b> ; handled by<br>commanding joints            |

- 3) only then the joint move to home, with everything now below the fork.

On a fault the arm is deliberately *not* sent home. After an abort it may be somewhere a joint move cannot leave safely, and moving is how a small problem becomes a crash; the program stops where it is and asks for hand-guidance.

#### G. API Migration

Table VI records the mapping from the original Python API to the Lua one. Only the last row is conceptually interesting: the original program selected the arm configuration explicitly, and the replacement has no such call, which is precisely the gap that joint-space commanding fills.

Two environment-specific facts cost time and are recorded so they are not relearned. Motion commands bind only in the main thread's script file; placed in the shared globals file they raise attempt to call a nil value, while ordinary Lua such as `print` works in both, which makes the failure confusing. And the joint-move point is a table keyed `joint`, not four positional arguments and not the `coordinate` key that the Cartesian move uses, an asymmetry in the vendor API rather than in the program.

## IV. COLLABORATIVE ROBOT TRANSFER

The Pro600 is a six-degree-of-freedom collaborative arm with 600 mm reach and 3 kg payload. Where the SCARA is confined to a horizontal plane with a vertical screw, the Pro600 can present the tool at an arbitrary orientation, which is what lets it approach the cradle from above and the deposit platform at three distinct positions.

The arm is programmed in RoboFlow, a block-structured environment hosted on the controller's own Linux system



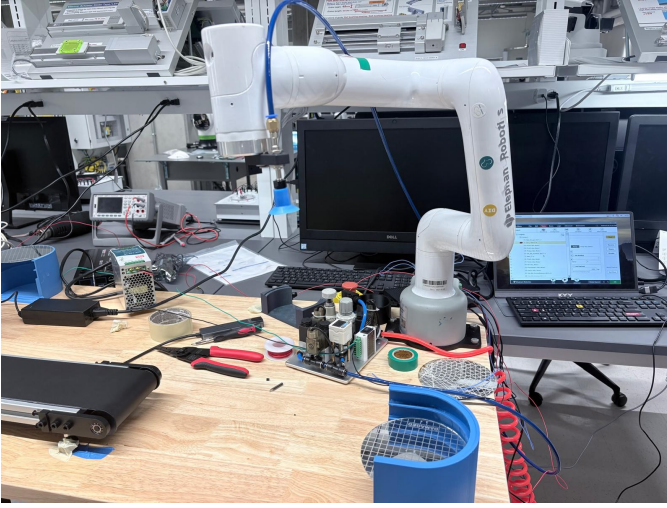


Fig. 5. The Pro600 at its taught home pose with the vacuum tool fitted. The pneumatic manifold is on the bench in front of the base; RoboFlow runs on the laptop at right over the controller’s remote desktop.

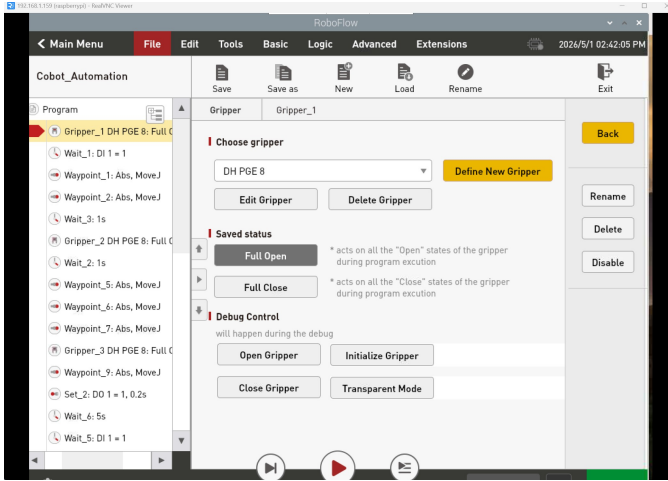


Fig. 6. The RoboFlow program tree for the transfer cycle. The step list at left interleaves absolute joint moves with tool actuations, a wait on DI 1, and a pulse on DO 1; the arm’s whole interface to the rest of the cell.

and reached over remote desktop (Fig. 6). The cell program alternates absolute joint moves with tool actuations and two I/O blocks that constitute its entire integration surface: a `Wait` on digital input 1, and a `Set` that pulses digital output 1. A Python path over `pymycobot` exists as an alternative and expresses the same cycle as an explicit list of Cartesian waypoints with per-segment speeds, three deposit positions cycled in turn, and the same input-wait / output-pulse handshake.

#### A. End Effector

The original build used a DH PGE 8 electric parallel gripper with 3D-printed finger extensions that added edge-contact geometry the flat stock pads could not provide. The present build uses a vacuum tool instead (Fig. 7). The change is not merely a swap of hardware: a parallel gripper is a two-state device the controller commands directly, whereas the vacuum

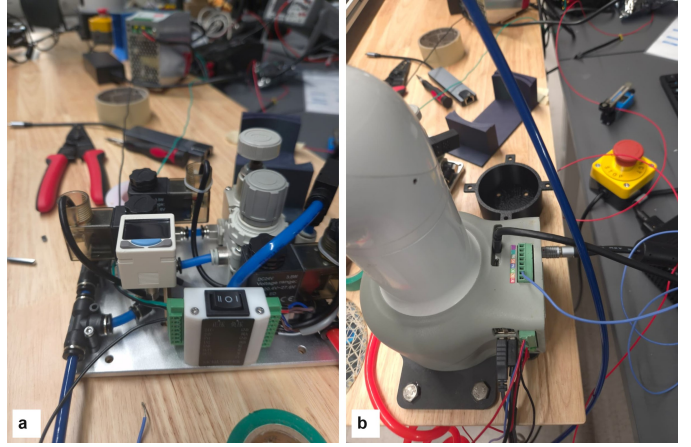


Fig. 7. (a) Vacuum circuit for the cobot tool: filter-regulator, two 24 V solenoid valves, digital vacuum switch and a positive/negative pressure control board. (b) The Pro600 base I/O terminal block, showing the labelled IN1 to IN6 points used for the cell handshake.

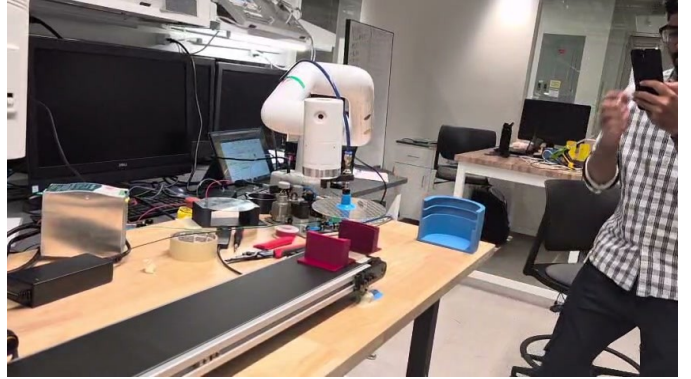


Fig. 8. The Pro600 depositing a wafer on the rotating platform at the end of a transfer, with the loaded cradle visible on the belt.

tool is a pneumatic circuit: a filter-regulator, two 24 V solenoid valves for vacuum and blow-off, and a digital vacuum switch whose output is the only evidence that a wafer is actually held. The practical consequence is that grip confirmation becomes a sensed quantity rather than an assumed one, which is a strict improvement for a fragile part, at the cost of a compressed-air dependency the gripper did not have.

## V. CONVEYOR TRANSPORT AND INSPECTION

### A. Drive and Sensing

The belt is driven by a stepper motor through the Dobot Magic Box extended I/O module, which exposes both a motor port and general-purpose digital I/O through the same Python API that drives the arm: one connection, one library, two subsystems. Direction is the sign of the commanded speed and stopping is a commanded speed of zero, so no mechanical changeover is involved.

Two infrared photoelectric sensors detect the cradle at each end of travel. Both are NPN devices with active-low outputs: the signal falls to 0 V on detection and returns high when

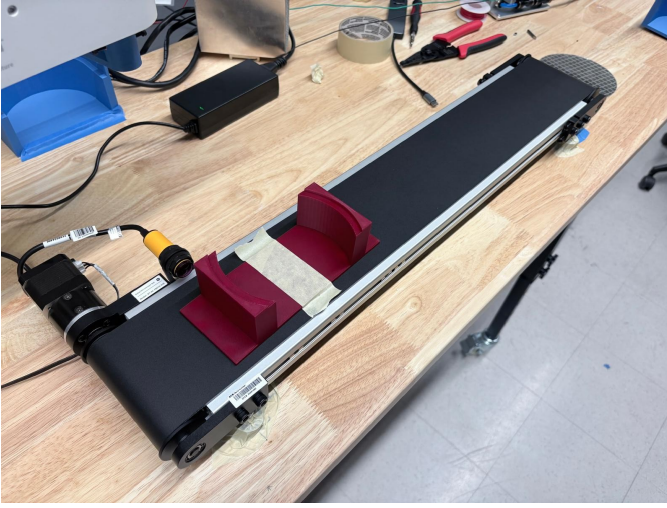


Fig. 9. The conveyor. Stepper motor at left, infrared photoelectric sensor in the yellow barrel housing, and the 3D-printed wafer cradle taped to the belt. The wafer cassette is at the rear left and a wafer is visible at the far end of the belt.

TABLE VII  
CONVEYOR TRANSITIONS

| Trigger             | Belt state | Action                      |
|---------------------|------------|-----------------------------|
| Load-end sensor low | stopped    | run forward; arm the camera |
| Load-end sensor low | reversing  | stop; ready for next wafer  |
| Pick-end sensor low | forward    | stop; cobot picks here      |
| Pick-end sensor low | stopped    | run reverse; return belt    |

clear. They are configured as digital inputs on the Magic Box before being read.

### B. Belt State Machine

Belt motion is governed by the pairing of sensor state with current belt state, given in Table VII. Encoding the transition on *(sensor, belt state)* rather than on sensor alone is what allows the same sensor to mean “depart” on one pass and “stop” on the return.

### C. Vision Inspection

The inspection stage converts each frame to greyscale, applies a median blur to suppress speckle from the belt texture, and runs a circular Hough transform tuned to the wafer’s apparent radius. On the first confirmed detection of a cycle the belt is stopped, a settling delay is allowed, a frame is captured and written to a sequentially numbered file, and the belt restarts.

Two latches make the stage behave sensibly in a continuous loop. A *first-run* flag suppresses detection until a wafer has actually been loaded, which prevents the transform from triggering on circular background objects before the cycle begins. An *already-photographed* flag limits capture to one image per wafer and is cleared when the load-end sensor fires again. Without them the same wafer is photographed repeatedly while it sits in view, and the belt stops each time.

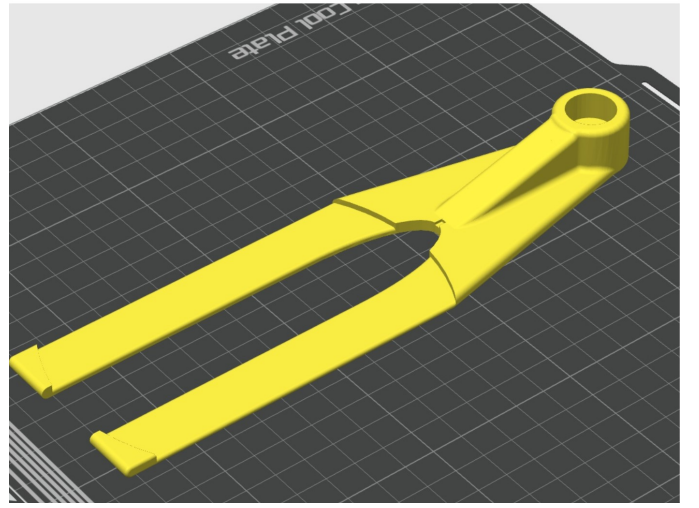


Fig. 10. The SCARA fork as prepared for manufacture: two thin prongs on a tapered yoke, with a central mounting boss for the  $J_4$  flange. The prongs must fit inside a cassette slot without touching its walls, and the tapered tips ease entry when the fork is a little off-centre. This is the one printed part carried over unchanged; Fig. 3 shows it fitted to the arm.

The transform is used as a presence and gross-geometry check rather than as a defect classifier; a genuine inspection function would need controlled illumination and a calibrated scale, neither of which the bench provides.

## VI. FIXTURE AND END-EFFECTOR DESIGN

Four families of part are printed for this cell: the SCARA fork, the cobot tool adapter, the multi-shelf cassette, and the wafer cradle that rides the belt. The fork is shown as prepared for manufacture in Fig. 10; the cassette and the cradle were both redesigned after that model was made, and the versions actually in service are the ones photographed in Fig. 4 and Fig. 9.

The two gripping strategies represent a genuine design trade rather than a preference. *Under-support* places no force on the wafer at all and is insensitive to small positioning errors in the plane, but it needs clearance under the part and it cannot hold the wafer against acceleration, which bounds how fast the SCARA may move. *Edge or face acquisition*, the parallel gripper in the original build and vacuum in the present one, holds the wafer positively and permits faster motion, but it demands accurate approach and it applies load to the part. Placing under-support at the cassette, where clearance exists and the wafer starts at rest, and positive acquisition at the cradle, where the wafer must be lifted clear of a constraining fixture, assigns each strategy to the station that suits it.

The cassette shelf pitch propagates into the SCARA program through (5) and is the single geometric parameter that every cycle after the first depends on. A design that fixes the pitch in the printed part, and a program that refuses to run a multi-wafer plan until the pitch is confirmed, are two halves of the same safeguard.

TABLE VIII  
AS-BUILT MICRO820 I/O ASSIGNMENT

| Point | Alias           | Role                              |
|-------|-----------------|-----------------------------------|
| DI_06 | Start_Switch    | start pushbutton                  |
| DI_07 | Stop_Switch     | stop; measured normally closed    |
| DI_04 | SCARA_DO        | SCARA cycle-done; read inverted   |
| DI_05 | Cobot_DO        | cobot cycle-done; read direct     |
| AI_00 | IR_SCARA_End    | IR sensor, cassette end; WORD     |
| AI_01 | IR_Cobot_End    | IR sensor, cobot end; WORD        |
| DO_04 | DI_to_SCARA     | 2s trigger pulse to the SCARA     |
| DO_05 | DI_to_Cobot     | 2s trigger pulse to the cobot     |
| DO_00 | Conv_SCARA_Side | conveyor, negated coil            |
| DO_01 | Conv_Cobot_Side | conveyor, negated coil            |
| AO_00 | Conveyor        | provisioned; unused by the ladder |

A three-dimensional model of the operator panel and controller was also built in `three.js` as a browser-hosted visual aid for I/O familiarisation before working on the live panel.

## VII. PLC COORDINATION

### A. Why a PLC Here

The PLC’s justification in this cell is determinism of scan, not computing power. A Python script on a PC is subject to the operating system’s scheduler, to garbage collection and to whatever else the machine is doing; a PLC executes its program on a bounded, repeatable scan. The Micro820 is therefore given exactly the job that needs that property: watching sensor zones, emitting timed trigger pulses, latching done reports and hosting the operator interface. It is given none of the jobs that do not, and drives neither the belt motor nor either arm.

The controller is a 2080-LC20-20QBB with twelve digital inputs, seven transistor outputs, four analog inputs and one analog output (Fig. 11). Note the two isolated output groups: O-00 through O-03 share CM0 while O-04 through O-06 share CM1, which is what permits the conveyor-side outputs and the robot-trigger outputs to sit in separate reference domains.

### B. I/O Map

Table VIII is the as-built assignment, recovered from the project’s global variable table (Fig. 12) and cross-checked against the ladder source. Aliases, not raw addresses, appear on the rungs, and getting them right is most of what makes the ladder readable.

Two details in Table VIII are easy to pass over and expensive to get wrong. The two infrared sensors are on *analog* inputs, typed *WORD*, so detection is a numeric comparison against a threshold variable rather than a bit test; and the two robot done-signals are read with *opposite* contact forms, because the SCARA’s NPN output pulls low on assertion while the cobot’s output drives high.

### C. The As-Built Ladder

The program is seven rungs (Fig. 13), summarised in Table IX.

Three design choices in this ladder are worth naming.

TABLE IX  
RUNG-BY-RUNG FUNCTION OF THE AS-BUILT PROGRAM

| # | Function                                                                                        |
|---|-------------------------------------------------------------------------------------------------|
| 1 | Stop in series with (Start OR Start) $\rightarrow$ Start. Seal-in latch.                        |
| 2 | Start $\wedge$ (AI_00 $\leq$ threshold) $\rightarrow$ TP 2s $\rightarrow$ DI_to_SCARA.          |
| 3 | Start $\wedge$ SCARA_DO $\rightarrow$ TP 2s $\rightarrow$ SCARA_TaskVar.                        |
| 4 | Start $\wedge$ (DI_to_SCARA $\vee$ SCARA_TaskVar) $\rightarrow$ <i>negated</i> Conv_SCARA_Side. |
| 5 | Start $\wedge$ (AI_01 $\leq$ threshold) $\rightarrow$ TP 2s $\rightarrow$ DI_to_Cobot.          |
| 6 | Start $\wedge$ Cobot_DO $\rightarrow$ TP 2s $\rightarrow$ Cobot_TaskVar.                        |
| 7 | Start $\wedge$ (DI_to_Cobot $\vee$ Cobot_TaskVar) $\rightarrow$ <i>negated</i> Conv_Cobot_Side. |

*Pulse timers, not on-delay timers.* The timers are IEC TP instances with a two-second preset. A TP produces a fixed-width output pulse on a rising input and is immune to what the input does for the rest of that pulse; an on-delay timer would instead track its input and release as soon as the input fell. Since the whole point of these rungs is to emit a trigger of guaranteed width to a robot that samples its input periodically, TP is the correct primitive.

*Two-second pulses are a deliberate margin.* A Micro820 scan is on the order of milliseconds. A two-second assertion is three orders of magnitude longer, so a level test on the receiving side cannot miss it and no edge capture is needed anywhere in the cell.

*Negated coils on the conveyor outputs.* DO\_00 and DO\_01 idle *high* and drop low for the duration of the pulse. A loss of PLC power therefore de-energises the outputs into the same state as an active command, which is safe only if the belt driver treats the de-energised state as stop. This is the fail-safe convention, and it is also a trap for anyone reusing these points for a driver with the opposite sense; the sequencer of Section VII-H carries an explicit note and an inversion at that boundary.

### D. Analog Thresholding

The infrared sensors return a voltage that falls when a wafer enters the detection zone, digitised to raw counts spanning 0 to 4095 for 0 to 10 V. The threshold lives in a *UINT* variable with a default of 1000 and is compared with  $\leq$  on rungs 2 and 5. It is calibrated by observing the clear and obstructed readings on a live watch window and choosing a value between them, closer to the obstructed reading.

The threshold is **not retained**. It reverts to its default on every project download and must be re-entered afterwards. Because the failure is silent (the cell simply never triggers, or triggers continuously), this is recorded here as a standing operational hazard rather than a footnote.

The analog output is aliased and provisioned but is not referenced by any rung in the as-built program; its stored setpoint of 1323 counts corresponds to approximately 3.2 V.



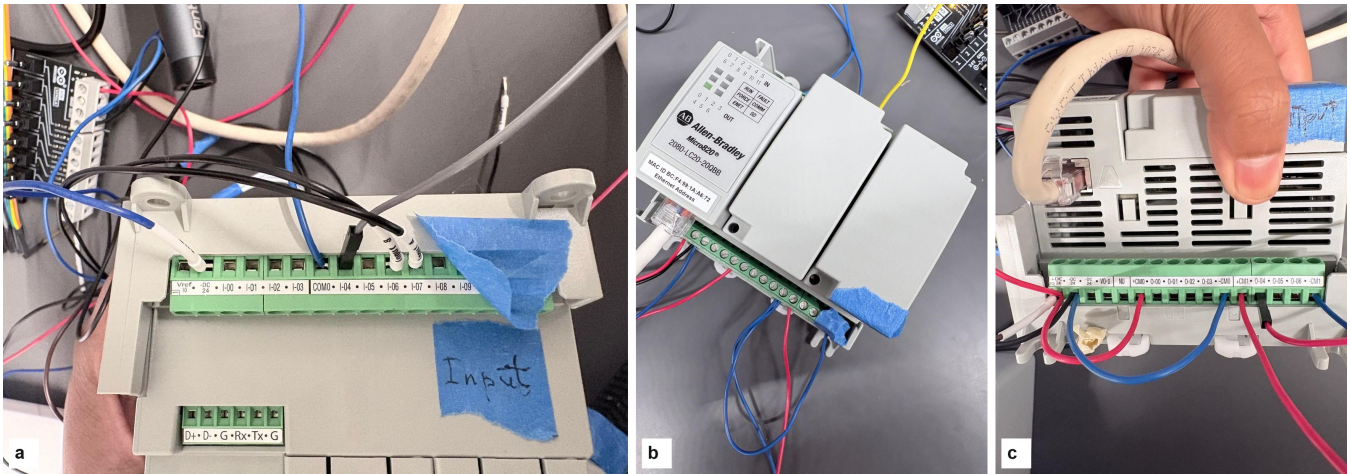


Fig. 11. Micro820 hardware as wired. (a) Input terminal block:  $V_{ref}/-DC24$ , I-00 to I-03, then COM0 and I-04 to I-09; the two labelled leads land on I-06 and I-07. (b) The 2080-LC20-20QBB module with its Ethernet lead and status LEDs. (c) Output terminal block: +CM0, O-00 to O-03, -CM0, then a second isolated group +CM1, O-04 to O-06, -CM1.

| Name              | Alias           | Data Type | Dimension | Project Value | Initial Value | Comment       | Retained                 | String Size |
|-------------------|-----------------|-----------|-----------|---------------|---------------|---------------|--------------------------|-------------|
| _IO_EM_DI_00      |                 | BOOL      |           |               |               |               | <input type="checkbox"/> |             |
| _IO_EM_DI_01      |                 | BOOL      |           |               |               |               | <input type="checkbox"/> |             |
| _IO_EM_DI_02      |                 | BOOL      |           |               |               |               | <input type="checkbox"/> |             |
| _IO_EM_DI_03      |                 | BOOL      |           |               |               |               | <input type="checkbox"/> |             |
| _IO_EM_DI_04      | SCARA_DO        | BOOL      |           |               |               |               | <input type="checkbox"/> |             |
| _IO_EM_DI_05      | Cobot_DO        | BOOL      |           |               |               |               | <input type="checkbox"/> |             |
| _IO_EM_DI_06      | Start_Switch    | BOOL      |           |               |               |               | <input type="checkbox"/> |             |
| _IO_EM_DI_07      | Stop_Switch     | BOOL      |           |               |               |               | <input type="checkbox"/> |             |
| _IO_EM_DI_08      |                 | BOOL      |           |               |               |               | <input type="checkbox"/> |             |
| _IO_EM_DI_09      |                 | BOOL      |           |               |               |               | <input type="checkbox"/> |             |
| _IO_EM_DI_10      |                 | BOOL      |           |               |               |               | <input type="checkbox"/> |             |
| _IO_EM_DI_11      |                 | BOOL      |           |               |               |               | <input type="checkbox"/> |             |
| _IO_EM_DO_00      | Conv_SCARA_Side | BOOL      |           |               |               |               | <input type="checkbox"/> |             |
| _IO_EM_DO_01      | Conv_Cobot_Side | BOOL      |           |               |               |               | <input type="checkbox"/> |             |
| _IO_EM_DO_02      |                 | BOOL      |           |               |               |               | <input type="checkbox"/> |             |
| _IO_EM_DO_03      |                 | BOOL      |           |               |               |               | <input type="checkbox"/> |             |
| _IO_EM_DO_04      | DI_to_SCARA     | BOOL      |           |               |               |               | <input type="checkbox"/> |             |
| _IO_EM_DO_05      | DI_to_Cobot     | BOOL      |           |               |               |               | <input type="checkbox"/> |             |
| _IO_EM_DO_06      |                 | BOOL      |           |               |               |               | <input type="checkbox"/> |             |
| _IO_EM_AI_00      | IR_SCARA_End    | WORD      |           |               |               |               | <input type="checkbox"/> |             |
| _IO_EM_AI_01      | IR_Cobot_End    | WORD      |           |               |               |               | <input type="checkbox"/> |             |
| _IO_EM_AI_02      |                 | WORD      |           |               |               |               | <input type="checkbox"/> |             |
| _IO_EM_AI_03      |                 | WORD      |           |               |               |               | <input type="checkbox"/> |             |
| _IO_EM_AO_00      | Conveyor        | WORD      |           |               |               |               | <input type="checkbox"/> |             |
| AO_Set            |                 | WORD      |           |               | 1323          |               | <input type="checkbox"/> |             |
| test              |                 | BOOL      |           |               |               |               | <input type="checkbox"/> |             |
| Start             |                 | BOOL      |           |               |               |               | <input type="checkbox"/> |             |
| SCARA_TaskVar     |                 | BOOL      |           |               |               |               | <input type="checkbox"/> |             |
| Cobot_TaskVar     |                 | BOOL      |           |               |               |               | <input type="checkbox"/> |             |
| > _SYSVA_CYCLECNT |                 | DINT      |           |               |               | Cycle counter | <input type="checkbox"/> |             |

Fig. 12. Global variable table for the Micro820, cropped to the variable list. The alias column is the as-built I/O map. Note that AI\_00 and AI\_01 are WORD types, so the infrared sensors are read as raw counts and thresholded numerically rather than as booleans; that only DI\_04 to DI\_07, DO\_00, DO\_01, DO\_04, DO\_05 and the two analog inputs are claimed, leaving the rest free for the handshake program of Section VII-F; and that nothing in the table is marked retained, which is why the detection threshold is lost on every download.

### E. Divergence from the Published Program

The version of this program printed in the course manual, and the compiled text artefact that accompanies it, are *not* the program that runs. Table X lists the differences established by reading the ladder source directly and corroborating against the controller's own logical values export.

The consequence is operational, not academic. Retyping the

published listing into the workbench would regress the cell in two specific ways: it would remove the stop input entirely, and it would leave two robot-trigger rungs running whether or not the operator has pressed start. The project file, opened in the workbench, is the authority; the printed listing is a historical artefact.

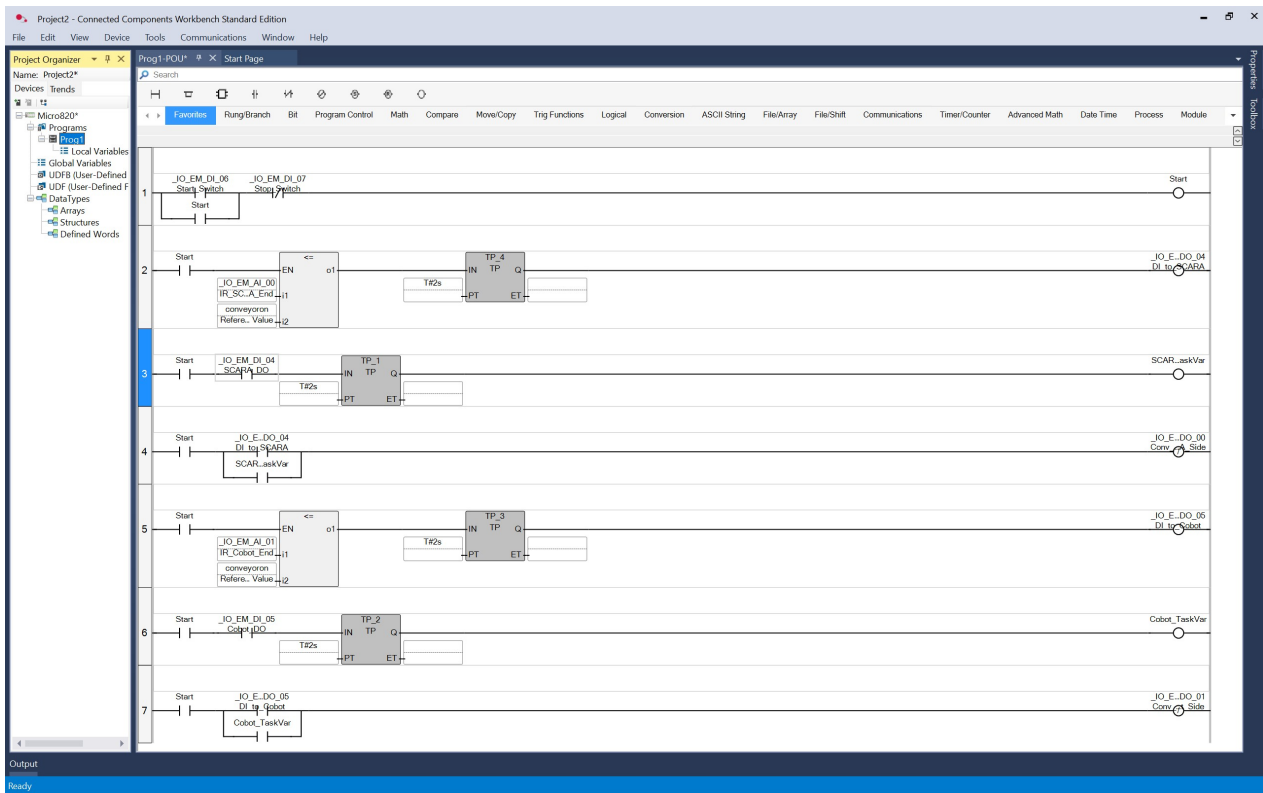


Fig. 13. The as-built seven-rung ladder in Connected Components Workbench. Rung 1 is the start/stop seal-in. Rungs 2 and 5 threshold the analog infrared sensors and fire 2 s pulse timers at the two robot triggers. Rungs 3 and 6 latch the two robots' done reports. Rungs 4 and 7 drive the conveyor outputs through *negated* coils, so those points idle high and drop low to act.

TABLE X  
PUBLISHED PROGRAM VERSUS AS-BUILT LADDER

| Aspect         | Published                  | As built                  |
|----------------|----------------------------|---------------------------|
| Stop input     | absent                     | DI_07, seal-in latch      |
| Rung gating    | two rungs ungated          | every rung gated on Start |
| Timers         | TON, six instances         | TP, four instances        |
| Naming         | TRIGERA..D                 | functional aliases        |
| Conveyor coils | described as normally high | negated coils             |

#### F. Handshake Rungs for the New SCARA

The as-built ladder addresses the original M1. The M1 Pro presents a different electrical interface: its outputs drive high through the opto board rather than pulling low. A dedicated three-rung program was therefore developed for the new arm, exercising one wafer per button press. Two wires carry the entire conversation:

Micro820 O-03 → M1 Pro DI10 “take a wafer”  
M1 Pro DO10 → Micro820 I-04 “wafer placed”

Both robot points are on the M1 Pro's *second* terminal block; the first block's silkscreen runs only to channel 8, which is visible in Fig. 14 and was confirmed with a motion-free probe script before either signal was trusted.

Expressed as structured text, the three rungs are:



Fig. 14. The M1 Pro controller I/O face. The upper terminal block carries DO1 to DO8 and DI1 to DI8; the lower, wider block carries DO9 to DO16 and DI9 to DI16. The handshake uses DI10 and DO10 on the second block.

```
(* R1: ask for a wafer *)
Scara_in := (Start OR Scara_in)
          AND Stop AND NOT cycle_done ;

(* R2: latch the arm's done report *)
cycle_done := (Robot_Done OR cycle_done)
             AND (Start OR Scara_in) AND Stop ;
```

```
(* R3: cycle-running indicator *)
Light := Scara_in ;
```

Rung 1 is a seal-in with two break conditions. The parallel contact on the output latches a momentary press; `cycle_done` releases it when the arm reports; the stop input releases it on demand. Rung 2's reset term is (*Start OR Scara\_in*) rather than `Scara_in` alone, and that choice is what makes a held button behave. Traced against a button pressed and held through the done pulse, the latch is held up by *Start* after `Scara_in` drops and clears only on release: one press, one wafer, whether the operator taps the button or leans on it. With the narrower reset term, a still-held button would immediately request a second cycle.

Three ordering properties of the exchange are load-bearing:

- **The PLC must drop its request on seeing the done pulse.** The arm waits for its start input to *fall* before it will consider another wafer, so that one long assertion cannot be read as several requests. If the rungs hold the request high, the arm finishes moving and then waits forever, indistinguishable from the outside from a completed run.
- **The start signal is active high by choice.** A de-energised or disconnected PLC output then reads as *no request*. Active low, which the original NPN-sourced convention would have suggested, would read a dead wire as “take another wafer, indefinitely”, which is the wrong direction to fail in.
- **The request is not a run permit.** The arm samples its start input once, at the top of a wafer, and never looks again. Dropping the request mid-cycle means “do not begin another”, and nothing more. Stopping a moving arm is the emergency stop's job, and the report is explicit about this because the button's label invites the opposite assumption.

### G. Stop Button Polarity

The panel stop button was measured and found **normally closed**: its input reads true at rest and false when pressed. This is the fail-safe convention (a severed wire reads identically to a press, and the cell stops), but it inverts the contact form on every rung that references it. The stop therefore appears as a normally-open contact, which reads wrongly until it is said aloud: *conduct while the stop button is not pressed*.

The cost of getting this wrong was one download. The first version used a normally-closed contact, which is open at rest, so rung 1 could not conduct at all: pressing start did nothing whatever, with every other element of the logic correct. The general lesson is worth extracting. A rung that refuses to make power with every other contact showing true indicts the one contact whose polarity was assumed rather than measured.

### H. Cell Sequencer

Coordinating a full multi-wafer run needs more state than a seal-in provides, so a structured-text sequencer was written as the successor to the handshake rungs. Figure 15 gives its states.

Two structural decisions are deliberate. First, *outputs are derived from the state, once, every scan*, rather than set inside the case branches; no state can then leave an output latched on its way out, and the belt is incapable of being energised in any state but the two travel states. Second, *both travel states carry a timeout* in addition to their sensor condition. A belt that runs and never trips its sensor is precisely the failure that grinds a wafer off the end of the conveyor, and a sensor-only transition has no answer to it.

The sequencer also documents a wiring dependency that must be resolved before it can run: as originally wired, the two infrared sensors report to the Magic Box, where the PLC cannot see them. The PLC could start the belt but could never learn when to stop it. Either the sensors move to PLC digital inputs, or the analog-threshold form of the as-built program is used instead.

## VIII. OPERATOR INTERFACE

Supervision is provided at two levels (Fig. 16). A physical trainer panel carries the maintained run switch, four input buttons, four indicator lamps and an analog meter, wired directly to the Micro820 terminal block; it is the interface that works when nothing else does. A software HMI adds start and stop controls, a wafer counter, and live lamps for the start input, both analog sensors, both robot triggers and both conveyor outputs.

The counter deserves a note about representation. On the arm side the wafer count exists as an integer, but the only channel to the PLC is a single discrete output pulsed once per wafer, so what actually crosses the boundary is a sequence of edges that the PLC re-integrates. Transferring the count as a number requires a Modbus holding register; the point at which that belongs is marked in the arm program and is left as future work.

## IX. RESULTS AND DISCUSSION

### A. State of Commissioning

The SCARA extraction half of the cycle (approach, three-stage insertion, lift, three-stage withdrawal, rise to clear height) runs correctly on hardware with the taught cassette parameters. The traverse and deposit half executes against the five taught joint poses of Table IV, all of which pre-validate on branch A within every joint limit. The handshake was brought up with the wafer count set to one, so that a run proves the two wires rather than emptying the cassette; the count is a single parameter and nothing else in the program depends on it.

The conveyor, its sensor state machine and the vision capture run as an independent Python process. The cobot cycle runs from RoboFlow against its own taught waypoints. The as-built Micro820 ladder runs online with live I/O and drives the supervisory HMI.

### B. Nominal Timing

Table XI collects the programmed dwells that set the cell's rhythm. These are configured constants, not measured cycle times; they bound the coordination overhead per wafer at

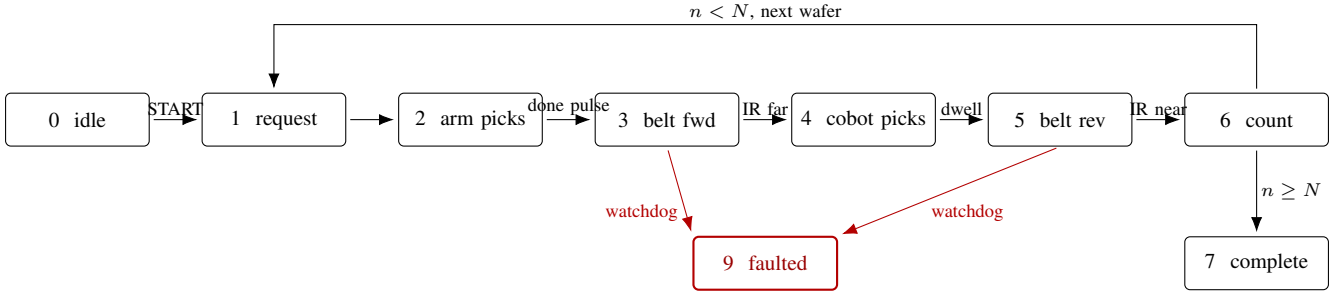


Fig. 15. Sequencer states for one wafer. Outputs are derived from the state once per scan rather than set inside the case branches, so the belt cannot be energised in any state but 3 and 5. Both travel states are bounded by a watchdog as well as by their sensor, because a belt that runs and never trips its sensor is the failure that carries a wafer off the end of the conveyor.

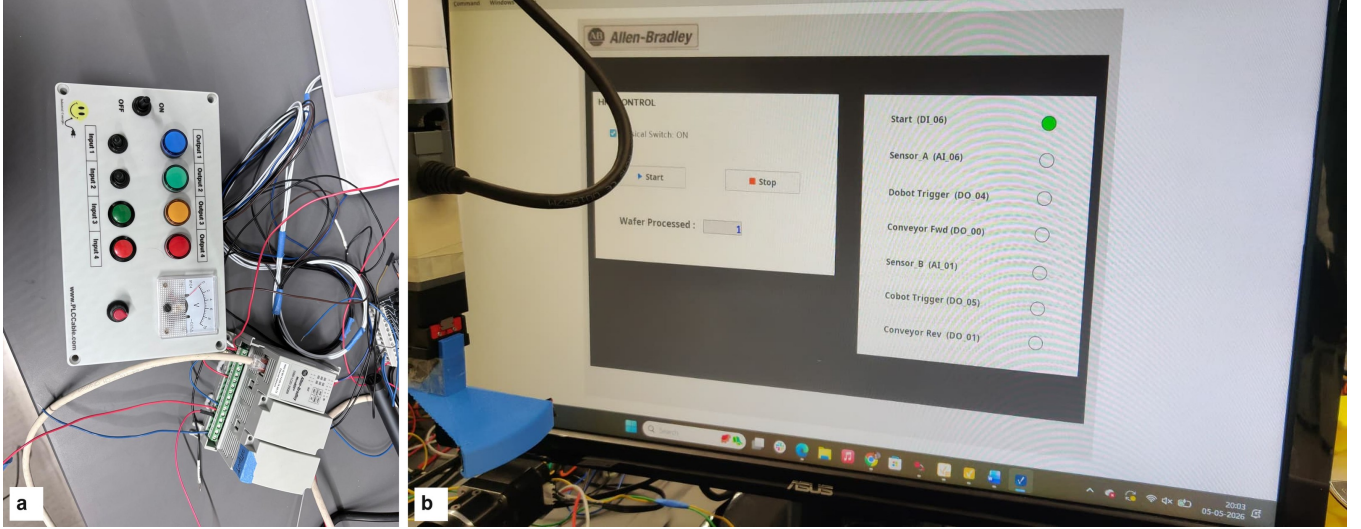


Fig. 16. (a) The physical operator panel wired to the Micro820: run switch, four inputs, four output lamps and a 0 to 10 V analog meter. (b) The supervisory HMI during a run, showing the start input asserted, the wafer counter at one, and the remaining I/O points idle.

TABLE XI  
PROGRAMMED TIMING CONSTANTS

| Constant                | Where              | Value  |
|-------------------------|--------------------|--------|
| Trigger pulse width     | PLC TP timers      | 2.0 s  |
| Cycle-done pulse width  | SCARA program      | 2.0 s  |
| Cobot done pulse        | RoboFlow Set block | 0.2 s  |
| Belt run/settle dwell   | conveyor script    | 2.5 s  |
| Camera settling delay   | conveyor script    | 2.0 s  |
| Standalone pacing dwell | SCARA program      | 3.0 s  |
| Cobot pick allowance    | sequencer          | 5.0 s  |
| Belt travel watchdog    | sequencer          | 20.0 s |

roughly seven seconds, on top of whatever the motion itself takes.

### C. Faults, Causes and Resolutions

Table XII records the substantive faults encountered, because in integration work the failure log is the result. Several of them share a shape: a signal was assumed rather than measured, and the resulting symptom pointed somewhere other than the cause.

### D. Discussion

Three observations generalise beyond this bench.

*Redundant representations catch errors that neither catches alone.* Every taught joint pose in Table IV is carried alongside its forward-kinematic Cartesian equivalent, computed by the program at startup rather than transcribed. Doing so caught the traverse pose taught at shelf height, and the mismatch between the approach and inserted heights, before either reached the hardware. The redundancy costs a few lines.

*The direction a signal fails in is a design parameter.* The start signal is active high so that a dead wire means no request; the conveyor outputs idle high so that loss of power stops the belt; the stop button is normally closed so that a cut wire reads as a press. None of these follows from the sensor convention; each is a decision about which failure the cell should default to, and each was made explicitly.

*Signal width should exceed the receiver's sampling uncertainty by a comfortable margin.* Two-second pulses against a millisecond scan mean no element of this cell needs edge detection. The cost is throughput, and on a laboratory cell that cost is worth paying; on a production cell the same



TABLE XII  
FAULTS ENCOUNTERED, ROOT CAUSES AND RESOLUTIONS

| Symptom                                             | Root cause                                                      | Resolution                                          |
|-----------------------------------------------------|-----------------------------------------------------------------|-----------------------------------------------------|
| Motion calls raise “nil value”                      | Robot API binds only in the main thread’s script                | Keep all motion in that file                        |
| “Incorrect motion point” at the fixture             | Only one IK branch is valid at 393.8 mm; solver chose the other | Command the fixture side in joint space             |
| Same, intermittently elsewhere                      | An edited parameter left a coordinate undefined                 | Check for nil before suspecting reach               |
| Cartesian jog cannot reach the fixture              | Jogging will not cross a configuration boundary                 | Expected; hand-guide, then teach joints             |
| Four cycles run for three wafers                    | Loop bounded by height, not by count                            | Derive $z_n$ from the count, (5)                    |
| Traverse waypoint descends to shelf height mid-path | Pose taught at the wrong height                                 | Raise $J_3$ only; pure screw move, branch unchanged |
| Fork dips 2.7 mm entering the slot                  | Approach and inserted poses taught at different heights         | Adopt the lower height for both                     |
| Wafer drags off the cradle                          | Deposit pose leaves prongs under the wafer                      | Add an explicit withdrawal pose                     |
| Start button does nothing at all                    | Stop wired normally closed but read as normally closed contact  | Invert the contact form                             |
| cycle_done latches at the press, not at the report  | Wrong contact form on the done input                            | Normally-open contact                               |
| Latch never releases                                | Seal-in rung with no break condition                            | Series break in the seal                            |
| Latch computed but unused                           | Nothing referenced the bit                                      | Consume it in both rungs                            |
| Robot-done input never true                         | The point had no alias; a local of the same name was read       | Alias the physical point                            |
| PLC absent from the browse tree                     | Link-local controller address                                   | Put the adapter on 169.254.x.x                      |
| Detection stops working after a download            | Analog threshold is not retained                                | Re-enter after every download                       |

argument would be made with edge capture and a handshake acknowledgement instead.

## X. LIMITATIONS AND FUTURE WORK

The cell as it stands has four acknowledged limitations.

*The deposit fixture is at 393.8 mm of a 400 mm reach.* Joint-space commanding makes the configuration deterministic, but it does not improve the conditioning of the workspace at that radius. Moving the fixture 30 to 40 mm towards the robot is the single change that would most reduce the cell’s fragility, and it costs one re-teaching session.

*Conveyor positioning is open loop.* The belt runs until a sensor sees the cradle. Position within the sensor’s detection window is uncontrolled, which bounds how tightly the cobot’s pick pose can be specified. A closed-loop approach, either an encoder on the drive or a second sensor for two-point edge timing, would tighten it.

*The wafer count crosses the cell boundary as edges, not as a value.* A Modbus holding register would carry it as an integer and would let the HMI display the arm’s own count rather than the PLC’s reconstruction of it.

*Interlocking is procedural rather than enforced.* The busy output that would keep the rest of the cell out of the arm’s workspace is implemented and wired through the same guarded output helper as everything else, but it is disabled by default because its pin assignment is assumed rather than measured. Enabling it, against a confirmed terminal, is the first safety improvement worth making.

Beyond these, vision-based wafer alignment, using the inspection camera to correct the cradle pose before the cobot picks, would remove the cell’s dependence on the cradle being taped in exactly the taught place, and is the natural next stage.

## XI. CONCLUSION

A five-station wafer handling cell was re-hosted onto substantially different robot hardware and returned to coordinated operation. The engineering content lay in two places. In motion, a fixture placed within 1.5% of the SCARA’s full reach admits only one inverse-kinematic branch, and a vendor API that exposes no configuration selector cannot be made to choose it from Cartesian commands; commanding that half of the cycle in joint coordinates carries the branch implicitly and resolves the failure, while the cassette half remains parametric Cartesian because a new height is required each shelf. In coordination, the as-built ladder was shown to differ from its published description in ways that matter for safety (a stop input, a start gate on every rung, pulse rather than on-delay timers), and a handshake was designed around the principle that each signal should fail in a stated direction. The faults encountered, catalogued in Table XII, are dominated by a single pattern: an electrical or geometric property assumed rather than measured, presenting as a symptom that pointed elsewhere.

## ACKNOWLEDGMENT

The wafer automation cell, its original commissioning and the accompanying laboratory manual [1] are the work of S. Chhabra, carried out under the guidance of Dr. Sangram Redkar in the RAS 598 Advanced Robotic Systems program at Arizona State University. The author thanks Dr. Redkar for access to the cell and for his guidance throughout this work.

## REFERENCES

- [1] S. Chhabra, *Wafer Automation Cell: Short Course Laboratory Manual*, RAS 598 Advanced Robotic Systems, Ira A. Fulton Schools of Engineering, Arizona State University, Spring 2026.
- [2] Dobot, *Dobot M1 Pro Hardware User Guide*, V1.3. Dimensions, workspace envelope and joint ranges.
- [3] Dobot, *DobotStudio Pro User Guide (MG400 & M1 Pro)*, V2.7, Appendix C: Script Commands.
- [4] Dobot, *Dobot M1 User Guide*, V1.5.3, and *Dobot M1 API Description*.
- [5] Elephant Robotics, *Pro600 Product Documentation and RoboFlow Software Guide*. [Online]. Available: <https://docs.elephantrobotics.com/docs/pro600-en/>
- [6] Elephant Robotics, *pymycobot Python API*. [Online]. Available: <https://github.com/elephantrobotics/pymycobot>

- [7] Rockwell Automation, *Micro820 Programmable Controllers User Manual*, publication 2080-UM002.
- [8] Rockwell Automation, *Connected Components Workbench Software*.
- [9] IEC 61131-3: *Programmable Controllers, Part 3: Programming Languages*, International Electrotechnical Commission.
- [10] OpenCV, *Hough Circle Transform*, `HoughCircles` reference. [Online]. Available: [https://docs.opencv.org/4.x/dd/d1a/group\\_\\_imgproc\\_\\_feature.html](https://docs.opencv.org/4.x/dd/d1a/group__imgproc__feature.html)
- [11] DH Robotics, *PGE Series Electric Parallel Gripper Documentation*.